

Milestone 1

Problem Definition & Literature Review

1. Define the objectives of the project:

Objective: To build a Clothing Assistant.

A chatbot designed to answer questions from store policy, previous purchases of customers, and suggest outfits based on query given and customer's previous purchases.

Part A: Store policy + Customer help desk

The chatbot will answer questions from store policies like what is the return policy, upcoming discount info, and so on. Additionally the customer can select a previously bought item and ask something like whether I can return or replace the product. Now the agent will check the return or replacement policy of that specific product and reply accordingly.

Part B: AI stylist

- The agent will be fed context about the customer preferences (previous purchases, size, etc). Without any query given the agent should concentrate on these preferences and suggest clothes. An additional advanced option could be suggesting clothes based on the skin tone of the customer if shared.
- Initially to get customer preferences and serve well a quiz will be designed.
- Given a query the agent should consider both customer preferences and query given and suggest clothes. (An example query would be 'suggest me an outfit for a graduation party')
- To prevent hallucinations the agent will only suggest products from the company catalogue. Also the agent should stick to the job of answering from company policy or suggesting outfits.
- The agent should have knowledge about fashion like which color shirt matches with which color pants, on which occasions what outfit is appropriate, etc,...

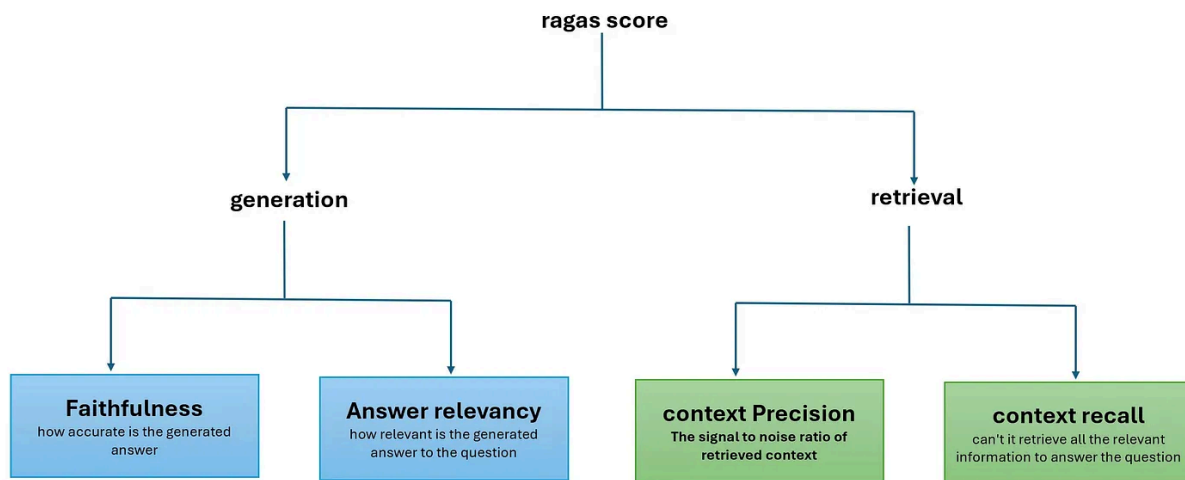
2. Review of existing solutions, baselines, and benchmarks:

Part A (Store policy + Customer help desk):

Scoring Metric:

Retrieval-Augmented Generation Assessment Suite (RAGAS)

RAGAS is a specialized evaluation framework for RAG systems. It goes beyond basic overlap metrics (BLEU, ROUGE) by focusing on how well the generated answer is grounded in retrieved evidence and how relevant / faithful it is.



The main components are:

Faithfulness: Measures how factually correct the generated response is, relative to the information in the retrieved documents.

$$\text{Faithfulness} = \frac{\text{Number of Correct Facts in the Response}}{\text{Total Number of Facts in the Response}}$$

Answer Relevancy: Assesses how well the answer addresses the original query (i.e. relevance of concepts used).

$$\text{Answer Relevancy} = \frac{\text{Number of Relevant Concepts in the Response}}{\text{Total Number of Concepts in the Response}}$$

Context Precision: Among the retrieved passages, how many are actually relevant to the query (i.e. proportion of useful retrieval).

$$\text{Context Precision} = \frac{\text{Number of Relevant Sentences Retrieved}}{\text{Total Number of Sentences Retrieved}}$$

Context Recall: Of all the potentially relevant sections, how many relevant ones were successfully retrieved.

$$\text{Context Recall} = \frac{\text{Number of Relevant Sentences Retrieved}}{\text{Total Number of Relevant Sentences Available}}$$

Composite Score:

$$\text{RAGAS} = \frac{F + R + P_{\text{ctx}} + R_{\text{ctx}}}{4}$$

This composite score offers a balanced measurement of how well our system retrieves supporting evidence and generates a relevant, factual answer in the domain of policy / FAQ documents. We will also analyze component scores individually to identify weaknesses (e.g. low context recall might indicate retrieval gaps, while low faithfulness might indicate hallucination in generation).

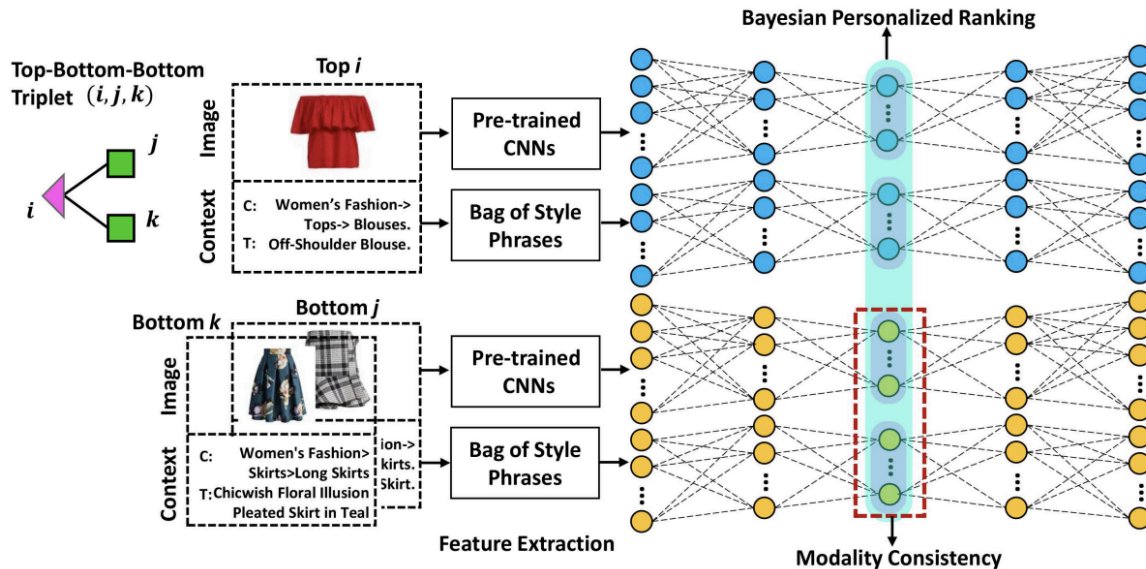
We will be aiming for a **RAGAS score of 0.8** or above which most RAG based applications are able to achieve. It has also been mentioned in the official RAGAS documentation:

<https://docs.ragas.io/en/stable/concepts/metrics/overview/#metric-design-principles>

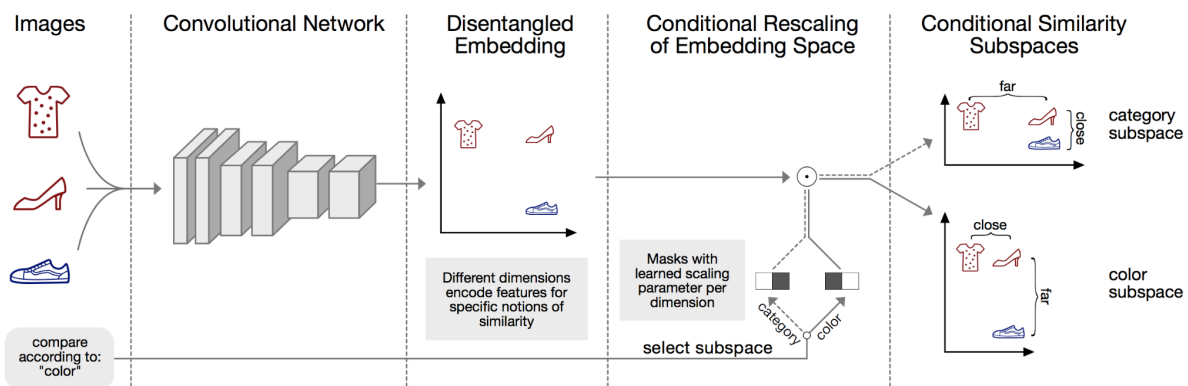
Part B (AI stylist):

Literature review:

1. NeuroStylist (<https://liqiangnie.github.io/paper/p753-song.pdf>)
 - a. Used image as well as bag of style text input
 - b. Trained to cluster matching top and bottom closer.
 - c. Used triplet (input + pair + non-pair) items for calculating loss
 - d. Used AUC and MRR metrics for performance



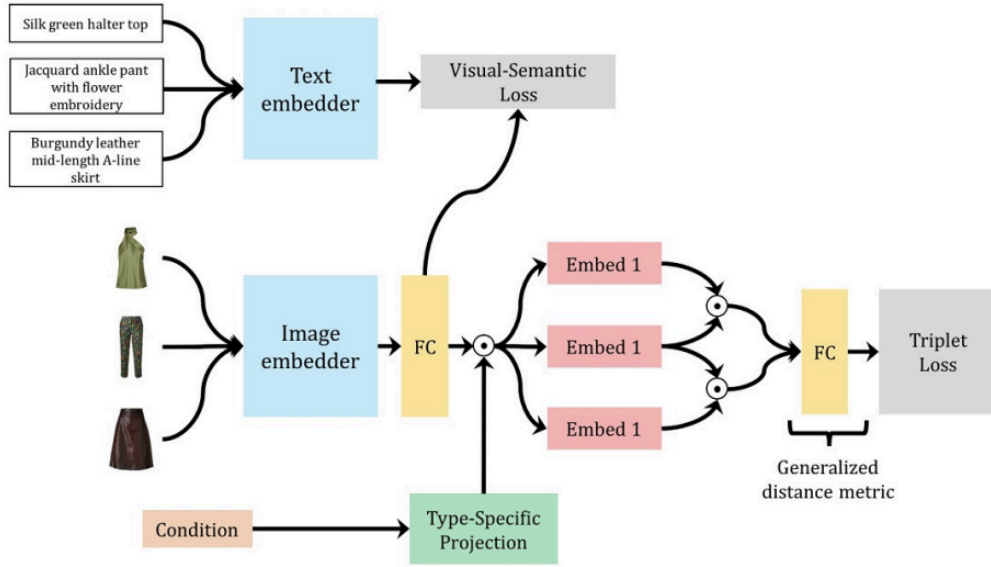
2. Conditional Similarity Networks (<https://github.com/andreasveit/conditional-similarity-networks?tab=readme-ov-file>)
 - a. Normally embedding the products clusters similar categories together.
 - b. This research paper focuses on clustering compatible products along a specific dimension



3. Learning Type-Aware Embeddings for Fashion Compatibility

(<https://arxiv.org/pdf/1803.09196>)

- Used both image and text for training the model
- Again triplets are used to make the model learn about compatibility



Inference from the literature reviews:

- All the papers we researched focused on suggesting a good pair given a top/bottom and some models also took into account user preference. But no model tried to take into account user defined filters during search.
- With respect to the dataset part, for filtering we require lots of features on which filters can be applied. The datasets which have features with it do not provide any compatible pairs data. If compatible pairs data is present, then features are not available.
- So, we are going to use the iMaterilist dataset, which has features with it. With respect to clustering compatible products using images, we won't be able to perform due to unavailability of compatible pairs data. Instead we will try to use text information alone and will build a suggestion system.

So, our suggestion system is going to use text features alone and will suggest clothes based on user inputs.

Scoring Metric (Tentative):

1. In the research papers, models trained would find a compatible top/bottom, but filters were not applied. But in our project filtering on user query is going to be a major part and so we will define a custom scoring metric.
2. Tentative scoring metric (for illustration)

$$\text{score} = 0.2 * \text{color} + 0.2 * \text{material} + 0.2 * \text{gender} + 0.2 * \text{category} + 0.2 * \text{compatibility}$$

3. Any filter that is not provided in the user query will be ignored during score calculation and we will scale the score based on the filters available.
4. For compatibility we experimented with metrics like color harmony, but if we get access to a trained model from the research sources we have referred to we may use that.

Model Performance Test (for Baseline and Benchmarks):

- a. Initially, we will not give the model much context about the nature of the data and specific instructions which it needs to follow for better search. We will use this as a baseline.
- b. Subsequently we will define steps to follow for better search and also will try to increase the llm's understanding of fashion (compatibility text based) and will compare the performance with the baseline.
- c. Additionally we will try different embedders and compare the performance.

Existing implementations based on which we identified gaps and scopes of improvement (Chatbot type implementations)

- **H&M “Virtual Stylist”**, experimental assistants / chatbots for style recommendations, virtual try-on etc.
- **Klarna’s “Shop-with-me / AI Style advisor”**, Style recommendations, curated product browsing, conversational interface in app / web.
- **Yesplz AI**. Quiz based model where a set of questions is framed and products are shown based on the answers. A similar kind of approach can be used for filtering the suggestions.
- **PairFashionExplanation** — Explains how prompt engineering can be used to get matching pairs of clothes. The dataset used is fashion VC which has image + description.

3. Data Sourcing, Licensing, and Embeddings:

For the **catalog**, we are considering a dataset containing all product details along with images, so that when a customer requests a particular item, the AI can search the dataset and display the corresponding product image. The initial thought process is we will embed the catalog and store it in a vector database for retrieval.

For **policies**, we will refer to the guidelines of major online clothing brands like Myntra and Bewakoof as baseline references, and then customize them to generate a comprehensive internal policy document for the chatbot.

Both the FAQ's and policy documents will be converted into embeddings using models like Sentence-BERT (for text), and stored in a vector database for fast and accurate retrieval during chatbot queries.

4. Gaps and Opportunities:

- Existing solutions relied on multiple external websites instead of a single catalogue, causing agents to suggest clothes outside of our catalogue. Whereas our AI will use only the company's internal product catalogue with all item details and images. Recommendations will be grounded in this dataset, ensuring items are available and from our catalogue. This prevents promoting third-party products.
- **The agents did not stick to the customer request** (when asked for a pant, the agents suggested other products too). So we will implement query-based filtering in the retrieval pipeline, using product metadata and embeddings to ensure the AI only selects items that match the customer's specific request.
- **The agents did not have much fashion sense.** Not always relevant products were shown. For example, when asked to suggest a pant for an already available red shirt, the agent did not pair a proper pant. Actually the pants suggested didn't even take into

account the red shirt. So we will create sets of rules for a GPT, so that outfit suggestions are context-aware and items like pants are paired appropriately with the user's existing clothing.

- **We didn't find any agents which requested for size** and filtered based on that too. The agents suggested a clothe and after the customer selects the product it showed sizes available. We will include size information in the product embeddings and apply size-based filtering during retrieval, so the AI only recommends items available in the user's preferred size.
- **The chat history is not maintained properly.** For example, we requested an outfit for a business meeting, and the agent suggested some blazers. Next we gave a prompt 'need a blue one'. Here the bot forgot about the occasion 'business meeting' and suggested even T-shirts. We will implement a vector-based conversational memory to retain context, ensuring follow-up queries consider previous prompts like occasion or item type.